

Lead Data Engineer - Consolidated Interview Questions

1. Can you walk us through how you would architect a data pipeline that handles both batch and real-time data from multiple on-prem and cloud sources using Azure services?

In my Azure implementations at KAHRAMAA and NAB, I designed a unified Lakehouse architecture using ADLS Gen2 as the storage layer and Databricks for processing. For batch ingestion from on-prem systems like Oracle or SQL Server, I used Azure Data Factory with Self-hosted Integration Runtime to securely extract data into ADLS. For real-time sources such as telemetry or API streams, I used Azure Event Hubs integrated with Databricks Structured Streaming. All raw data lands in the bronze layer, transformations happen in Silver using PySpark, and curated business datasets are exposed through Gold for Power BI and analytics. I ensure idempotency, schema enforcement, and governance via Delta Lake and Unity Catalog. Monitoring is centralized through Azure Monitor and Databricks logs. This approach supports both batch and streaming without duplicating business logic.

2. When designing real-time ingestion in Databricks, why would you choose Auto Loader? What latency and data volume can it typically handle?

I prefer Auto Loader because it is scalable, fault-tolerant, and optimized for incremental file ingestion in cloud storage like ADLS.

It tracks new files efficiently without relying on expensive directory listing operations.

In my implementations, Auto Loader combined with Structured Streaming handled high-volume structured and semi-structured ingestion reliably with near real-time latency typically seconds to a few minutes depending on micro-batch configuration and cluster size.

It supports schema evolution and checkpointing, which makes it production-safe and **replayable (Replayable means you can stop and restart the Auto Loader stream, and it will pick up exactly where it left off – processing only new files that arrived while it was stopped, without reprocessing old files or missing any.)**

. It's ideal when files are landing continuously rather than events being pushed directly via messaging systems.

3. Can you explain the Medallion architecture (Bronze, Silver, Gold layers) and how you would implement it in Databricks?

The Medallion architecture separates data into progressive layers of refinement.

Bronze contains raw, append-only ingested data with minimal transformation.

Silver contains cleansed, standardized, and validated data with business logic applied.

Gold contains aggregated, business-ready datasets optimized for reporting and analytics.

In Databricks, I implement this using Delta tables in ADLS, where each layer is physically separated and governed via Unity Catalog. Bronze captures schema drift, Silver enforces contracts and applies transformations, and Gold exposes curated metrics for dashboards. This layered approach improves reliability, replayability, and governance.

4. How would you set up multiple environments (Dev, QA, UAT, Prod) within a single Databricks workspace using Unity Catalog?

Within Unity Catalog, I would logically separate environments using catalogs and schemas. For example, dev_catalog, qa_catalog, and prod_catalog, each with its own schemas for Bronze, Silver, and Gold. Access control would be enforced via RBAC so that only appropriate users or service principals can write to production layers. CI/CD pipelines using Azure DevOps would promote notebooks and jobs across environments while preserving configuration differences. This ensures governance and separation of duties without requiring completely separate workspaces.

Because of this design, metadata is **not physically isolated** per environment when using a single workspace. The metadata for DEV, QA, and PROD will always reside in the same logical place

5. What are the differences between Self-hosted Integration Runtime, Azure Integration Runtime, and Auto-resolve Integration Runtime in Azure Data Factory?

Self-hosted Integration Runtime is installed on-prem and is used to securely connect to on-prem systems or private networks.

Azure Integration Runtime runs in Azure and handles cloud-to-cloud data movement and compute.

Auto-resolve Integration Runtime automatically selects the closest Azure region for optimal performance when moving data within Azure.

In my implementations, I've used Self-hosted IR for Oracle and on-prem SQL extraction, while Azure IR handled cloud-based transformations and movements efficiently.

6. When implementing real-time ingestion, how do you choose between Event Hubs, IoT Hub, Kafka, and Structured Streaming?

The choice depends on the source and protocol.

Event Hubs is ideal for high-throughput event ingestion in Azure-native architectures. IoT Hub is specialized for device telemetry and IoT scenarios.

Kafka is preferred when integrating with existing Kafka ecosystems or hybrid systems.

Structured Streaming is not an ingestion system itself but a processing engine that consumes from these messaging systems. In my experience, Event Hubs combined with Structured Streaming worked well for enterprise telemetry and transactional streams because it integrates seamlessly with Databricks and scales horizontally.

7. Suppose ADLS storage costs are rising even though data volume remains stable — how would you investigate this?

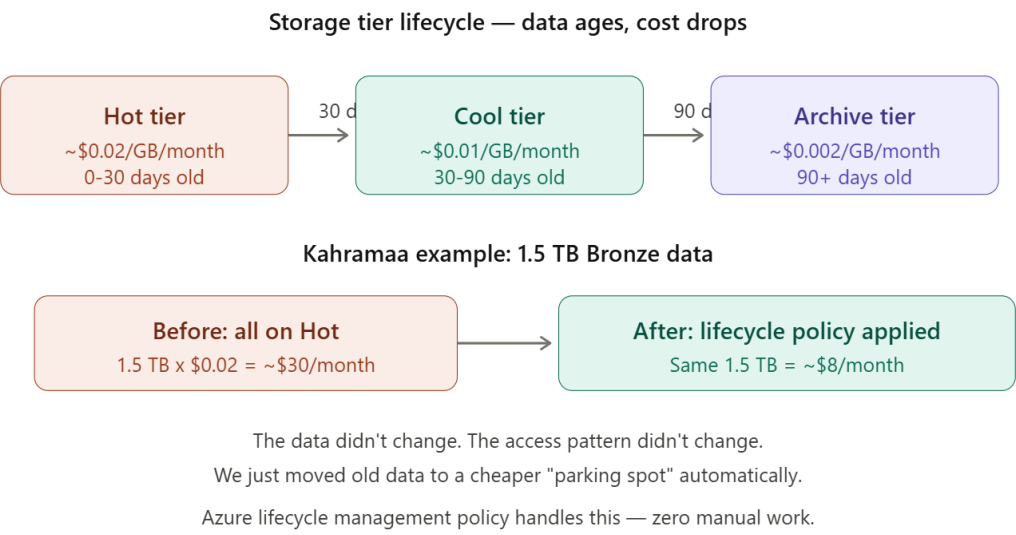
First, I would analyze storage metrics to check for small-file explosion or high transaction costs due to excessive file operations. Frequent MERGE operations can increase file counts even if logical volume is stable. I would also check retention policies and stale data accumulation in intermediate layers. In Delta environments, I verify whether VACUUM is being applied appropriately and whether historical versions are retained longer than necessary. Often, rising cost is due to write amplification or inefficient file management rather than true data growth.

This is something I've actually dealt with at Kahramaa. If ADLS costs are rising but data volume is stable, the first place I look is Delta Lake file versioning — and 90% of the time, that's the culprit.

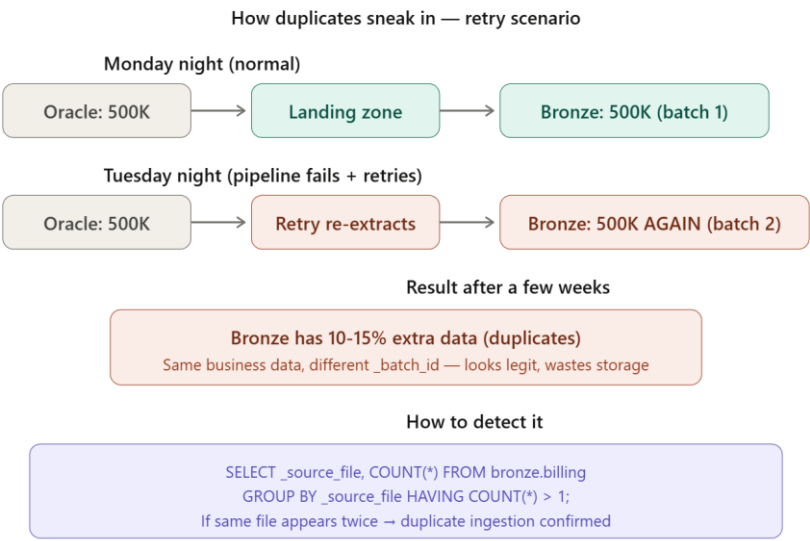
Delta Lake keeps old Parquet file versions every time you do an UPDATE, MERGE, or DELETE — that's how time travel works. But if **VACUUM isn't running regularly** or the retention is set too long, those old versions pile up fast. I've seen tables where the active data is 10 GB but the storage footprint is 80 GB because of months of accumulated versions. So the first thing I do is run DESCRIBE HISTORY on the suspect tables and check the detail() output to compare active file count and size against total storage. Then I run VACUUM RETAIN 168 HOURS DRY RUN to see how much would be reclaimed — often it's a dramatic number.

The second most common cause is the **small file problem**, especially on streaming tables. Our IoT pipelines run 30-second micro-batches, and each batch writes a small Parquet file. Over time, you get thousands of tiny files per partition — each with metadata overhead. I check file counts and average sizes per partition. The fix is OPTIMIZE to compact small files into larger ones, and I enable autoCompact and optimizeWrite on streaming tables so this happens automatically going forward.

Third, I check **storage tier configuration**. If all data — including Bronze data from six months ago that nobody queries — is sitting on the Hot tier, that's an easy win. Hot tier is roughly double the cost of Cool. I implement Azure lifecycle management policies that automatically move Bronze data to Cool after 30 days and Archive after 90.



Fourth, I check **for duplicate ingestion** — pipeline retries or misconfigured triggers causing the same files to land multiple times. I query Bronze for duplicate `_source_file` values. Auto Loader's file tracking prevents this, but if someone bypassed it with a manual load, duplicates creep in.



Fifth — and this one is often overlooked — Azure blob soft-delete. It keeps deleted blobs for a configurable retention period. If that's set to 365 days, every file you VACUUM'd is still being stored invisibly for a year. I review the soft-delete retention and snapshot policies.

At Kahramaa, the actual fix was a combination: we hadn't been running VACUUM consistently on some Silver tables, our streaming Bronze tables had a massive small file problem, and our lifecycle policy wasn't set up for Bronze. Once we addressed all three — scheduled VACUUM in our daily maintenance pipeline at 5:45 AM, enabled auto-compaction on streaming tables, and implemented tiering policies — storage costs dropped by about 35% within a month while the actual data volume stayed the same."

8. Can you explain Delta table transaction logs?

Delta transaction logs are stored in the `_delta_log` directory and track every change to a Delta table, including inserts, updates, deletes, and schema modifications. They enable ACID transactions, time travel, and rollback capabilities. Each commit is recorded as a JSON or checkpoint file, allowing consistent reads even during concurrent writes. In my projects, Delta logs provided reliability for auditability and historical data recovery, especially in regulated environments.

- Atomicity – All operations in a transaction succeed or none do.
- Consistency – Transaction brings database from one valid state to another.
- Isolation – Concurrent transactions do not interfere with each other.
- Durability – Committed changes persist even after a system failure.

9. What are Z-Ordering, OPTIMIZE, and VACUUM in Delta Lake, and how do they help with performance and cost control?

OPTIMIZE compacts small files into larger ones to improve scan efficiency. Z-Ordering clusters data based on frequently filtered columns, improving data skipping during queries. VACUUM removes obsolete data files that are no longer referenced in the transaction log, reducing storage cost. In my Databricks workloads, I

applied OPTIMIZE periodically to address small-file issues, used Z-Ordering strategically for high-cardinality filter columns, and configured VACUUM with retention policies aligned to compliance requirements. These operations improved query speed while controlling storage overhead.

10. If a Databricks job runtime doubled with only 1.5x data growth despite autoscaling, how would you diagnose and fix it?

I would first compare execution plans to see if partition pruning or broadcast joins were lost. Then I would check shuffle size, skew, and spill behavior in Spark UI. A small increase in data can disproportionately increase shuffle cost if partitioning is suboptimal. I would also examine file fragmentation and statistics freshness. In past scenarios, I've resolved similar regressions by optimizing file layout, adjusting partition strategies, enabling AQE, or tuning autoscaling limits. The key is diagnosing whether the issue is data layout, execution plan change, or resource saturation before applying fixes.

11. How do you handle partitioning strategies for large datasets?

I design partitioning based on query access patterns rather than blindly partitioning by every column. In my Databricks implementations, I typically partition large fact tables by date or ingestion date because most analytical queries filter on time ranges. For high-volume datasets, I avoid over-partitioning because too many small partitions lead to small-file issues. I also monitor partition distribution to prevent skew. Partitioning should balance write performance, read efficiency, and file management — not just theoretical distribution.

12. What is the difference between repartition and coalesce?

repartition reshuffles the entire dataset and can increase or decrease the number of partitions, which makes it suitable when you need even data distribution before joins or writes. coalesce reduces the number of partitions without a full shuffle and is more efficient when simply merging partitions, typically before writing fewer output files. In practice, I use repartition when fixing skew or preparing for joins, and coalesce when optimizing output file counts without needing redistribution.

13. How does predicate pushdown improve performance?

Predicate pushdown allows the query engine to filter data at the storage level before loading it into memory. In columnar formats like Parquet or Delta, metadata such as min/max statistics allows the engine to skip entire files or row groups. In my Delta Lake workloads, proper partitioning combined with predicate pushdown significantly reduced scan time, especially for time-based filters. It minimizes I/O and improves overall query efficiency.

14. If you have 1 TB of data and want to convert it into 5 output files, how would you do it?

In Spark, I would first ensure that the dataset is properly repartitioned to 5 partitions before writing. For example, using `df.repartition(5)` and then writing the data. If I simply want to reduce partitions without full shuffle and the data is already evenly distributed, I could use `coalesce(5)`. However, I would be cautious — writing only 5 files for 1 TB could create very large files, which might not be optimal for downstream processing. File size strategy should align with read patterns.

15. If you need to design a Spark cluster to process 1 TB of data, what parameters would you consider while designing it?

I would consider data distribution, shuffle size, partition count, executor memory, and CPU cores. I would estimate optimal partition sizing — generally targeting 100–250 MB per partition — to ensure efficient parallelism. I would configure executor memory to avoid spill and tune the number of executors based on workload characteristics. I also consider autoscaling limits and whether the workload is CPU-bound or I/O-bound. In my production environments, proper partition sizing and executor tuning were more impactful than simply increasing cluster size.

16. In this PySpark code `df.write.save('s3://...')`, what logical or syntax issues do you spot assuming we are writing to ADLS?

If we are writing to ADLS, using `s3://` is incorrect because that path refers to AWS S3. For ADLS Gen2, the correct format would be `abfss://container@storageaccount.dfs.core.windows.net/path`. Additionally, it's best practice to specify the format explicitly, such as `.format("delta")` if writing a Delta table. Not specifying format or path correctly can cause compatibility or storage issues.

If Partitioning by Date (Like You Did in Fact Tables)

```
df.write \
    .format("delta") \
    .partitionBy("reading_date") \
    .mode("append") \
    .save("s3://your-bucket-name/iot_fact/")
```

17. How do you implement data transformations in Databricks using SQL or PySpark?

In Databricks, I implement transformations either using Spark SQL for declarative transformations or PySpark for more complex logic. For structured business transformations, I often use SQL because it's easier to maintain and readable for analysts. For advanced transformations, joins, window functions, or UDF logic, I use PySpark. In my projects, I combine both approaches — SQL for aggregation and reporting layers, and PySpark for heavy transformation logic — ensuring modular and reusable code.

18. How do you implement checkpointing in Databricks pipelines?

Checkpointing is implemented in Structured Streaming by specifying a checkpoint location in cloud storage, such as ADLS. This directory stores offsets and progress metadata so the stream can resume safely after failures. In my streaming pipelines, I always configure a dedicated checkpoint path per job and ensure it is preserved during restarts. This guarantees exactly-once processing and replay safety.

19. Write a SQL query to create a column called `converted_sal` where the `sal` column is converted to dollar rate and the \$ symbol is displayed.

```
SELECT  sal, CONCAT('$', CAST(sal * 1.0 AS STRING)) AS converted_sal
FROM employees;

SELECT  CONCAT('$', ROUND(sal / 83.0, 2)) AS converted_sal
FROM employee;
```

20. Write a Python program to remove duplicates from a list.

```
def remove_duplicates(input_list):  
    return list(set(input_list))  
  
# Example  
  
nums = [1, 2, 2, 3, 4, 4]  
  
print(remove_duplicates(nums))
```

21. Solve a SQL window function problem.

```
SELECT *  
  
FROM (  
    SELECT *,  
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY transaction_date DESC) AS rn  
    FROM transactions  
) t  
  
WHERE rn = 1;
```

22. Write a SQL query to find the second highest salary.

```
SELECT sal FROM (  
    SELECT sal, DENSE_RANK() OVER (ORDER BY sal DESC) AS rnk  FROM employees  
) t  
  
WHERE rnk = 2;
```

23. Suppose you are given two tables containing 0s, 1s, NULL values, and duplicate records. How would INNER JOIN behave?

An INNER JOIN returns only rows where the join condition matches in both tables. NULL values do not match each other because NULL represents unknown. If duplicate keys exist in both tables, the result multiplies — meaning if one table has two matching rows and the other has three, the result will produce six rows for that key. I've seen this cause unexpected data explosion in joins, so I always validate key uniqueness before joining.

24. How would LEFT JOIN behave in that scenario?

A LEFT JOIN returns all rows from the left table and matching rows from the right table. If there's no match, right-side columns appear as NULL. Duplicate keys still cause row multiplication. If the join key in the right table is NULL, it won't match anything. This is commonly used when preserving primary dataset completeness is important.

25. How would RIGHT JOIN behave?

RIGHT JOIN is symmetrical to LEFT JOIN but preserves all rows from the right table. If no match exists in the left table, left-side columns become NULL. Like other joins, duplicates increase result size multiplicatively.

26. How would FULL JOIN behave?

FULL OUTER JOIN returns all rows from both tables. Matching rows are combined, while non-matching rows from either side appear with NULLs in the missing columns. It's useful for reconciliation scenarios, which I've used in financial comparisons between legacy and new systems.

27. How would CROSS JOIN behave?

A CROSS JOIN produces the Cartesian product of both tables — every row from the first table paired with every row from the second. This can explode exponentially and is rarely used intentionally in large-scale data engineering unless generating combinations deliberately.

28. How do NULL values impact join results?

NULL values do not equal other NULL values in standard SQL joins, so rows with NULL join keys typically do not match. In analytics pipelines, this can unintentionally drop records. I often handle this explicitly using COALESCE or by filtering before joins when NULL semantics must be controlled.

29. How do duplicate records impact the result set?

Duplicates multiply join output. If duplicates exist on both sides of a join key, results increase multiplicatively. This often leads to inflated aggregates. In my implementations, I either deduplicate upstream using DISTINCT or enforce uniqueness constraints in Silver layers to avoid downstream data inflation.

30. Can you explain the internal execution flow of a SQL query?

Logically, SQL executes in the order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. However, physically, the optimizer rewrites the query and chooses the most efficient plan based on statistics, indexes, and cost estimation. Understanding this difference is critical for performance tuning.

31. What stages are involved in generating an execution plan?

The stages include parsing (syntax validation), logical plan generation, query optimization (cost-based optimization and rule-based rewrites), and physical plan generation where join strategies, indexing, and parallelism are determined. In Spark or distributed systems, this also includes DAG generation and stage planning.

32. How do you interpret an execution plan for performance tuning?

I focus on identifying full table scans, large shuffle operations, skewed joins, missing partition pruning, and improper join strategies. In Spark UI, I examine stage duration, shuffle read/write size, and task skew. In my Databricks projects, comparing execution plans before and after changes helped isolate regressions effectively.

33. How does Microsoft Purview integrate for data cataloging, validation, and schema enforcement during ingestion?

Microsoft Purview integrates by scanning data sources like ADLS and Databricks to build a centralized data catalog. It captures lineage, schema metadata, and classification — including sensitive data detection. In enterprise setups, Purview enforces governance policies and helps track data movement across layers. I've used Purview to improve visibility and compliance tracking in Azure environments.

34. Can you explain the Unity Catalog setup including storage credentials, external locations, schemas, and PII masking?

Unity Catalog centralizes governance in Databricks. Storage credentials define secure access to ADLS. External locations map cloud storage paths to governed objects. Catalogs and schemas logically organize data by environment or domain. PII masking can be applied using column-level access policies, ensuring only authorized users can view sensitive columns. In my implementations, this setup enabled strict RBAC and auditability.

35. How do you ensure RBAC and permissions via Azure AD for Synapse and Power BI access?

I integrate Azure AD groups with Synapse and Databricks to control workspace-level and data-level access. In Power BI, I implement row-level security and control dataset access via Azure AD roles. Access is granted based on principle of least privilege. This ensures governance consistency across analytics platforms.

36. What is Slowly Changing Dimension Type 2 (SCD2)?

SCD Type 2 tracks full historical changes by inserting new rows whenever an attribute changes while maintaining effective start and end dates and an active flag. It preserves history rather than overwriting data.

37. How does SCD2 differ from other SCD types?

Type 1 overwrites data with no history. Type 2 maintains full history using new rows. Type 3 stores limited history by adding extra columns. SCD2 is preferred when historical traceability is required.

38. In which business scenarios would you implement SCD Type 2?

SCD2 is ideal for tracking changes in customer addresses, product attributes, compliance records, pricing history, or regulatory reporting especially in financial or audit-sensitive environments where historical accuracy matters.

39. Can you explain a practical example where you implemented SCD2?

In one of my Databricks implementations, I built SCD2 logic for tracking network configuration changes and business entity updates. Using Delta MERGE statements, I updated existing records by marking them inactive and inserted new rows with updated attributes and effective timestamps. Partitioning and Z-Ordering ensured query performance despite growing historical volume.

40. Explain normalization concepts including 1NF, 2NF, and 3NF.

1NF requires atomic columns and no repeating groups.

2NF ensures all non-key attributes fully depend on the primary key.

3NF eliminates transitive dependencies so non-key attributes depend only on the primary key.

In data engineering, I apply normalization in transactional systems but often denormalize in analytics layers to improve query performance.

41. What problems does normalization solve?

Normalization eliminates redundancy, improves data integrity, and prevents update anomalies. In transactional systems, it ensures that data is stored logically without duplication, reducing inconsistencies. In my early Oracle-based systems and warehouse integrations, normalization helped maintain clean relational structures before transforming data into analytical models.

42. What is denormalization?

Denormalization is the process of combining related tables to reduce joins and improve read performance. It introduces some redundancy but improves query speed, which is beneficial for analytical workloads.

43. In what scenarios would you prefer denormalization over normalization?

In analytics and reporting systems where query performance is critical, I prefer denormalization. For example, in Synapse and Databricks Gold layers, I model denormalized fact tables to reduce join complexity for Power BI dashboards. Normalization is ideal for transactional systems, while denormalization suits analytics.

44. What is a self-join?

A self-join is when a table is joined to itself using aliases to compare rows within the same table.

45. In what real-world scenarios would you use a self-join?

Self-joins are useful for hierarchical relationships such as employee-manager mappings or product category hierarchies. I've used self-joins in reporting scenarios where parent-child relationships were stored within a single table.

46. What is the difference between fact and dimension tables in a data warehouse?

Fact tables store measurable events such as transactions or metrics, typically with foreign keys to dimensions. Dimension tables store descriptive attributes such as customer details or product information. In my warehouse implementations, fact tables handle volume, while dimensions provide context.

47. How do you decide whether a table should be modeled as a fact or a dimension?

If the table contains quantitative, transactional data that grows continuously, it is modeled as a fact. If it contains descriptive attributes that provide context and change less frequently, it is modeled as a dimension. I base this decision on business granularity and query requirements.

48. Describe Star Schema vs Snowflake Schema.

Star Schema has a central fact table connected to denormalized dimension tables. Snowflake Schema normalizes dimensions into multiple related tables. I typically implement Star Schema for analytics because it simplifies queries and improves performance.

49. When have you implemented Star Schema (for example, in Synapse)?

In my Azure Lakehouse projects, I implemented Star Schema in curated layers for reporting. Fact tables stored transactional data, and dimensions represented entities such as customers and business units. This design improved Power BI performance and simplified business reporting.

50. How do business users access curated Gold layer data from Synapse to Power BI?

Curated Gold tables are exposed via Synapse SQL endpoints or Databricks SQL. Power BI connects using secure Azure AD authentication. I ensure RBAC policies and row-level security are enforced so users access only relevant datasets.

51. What is a watermark column?

A watermark column is typically a timestamp or incrementing value used to track the latest processed record during incremental loading.

52. How is a watermark used in incremental data loading?

During each load, I store the maximum processed watermark value. The next extraction fetches records where the watermark column is greater than the last processed value. This avoids reprocessing unchanged data.

53. How would you design an end-to-end pipeline using a watermark strategy?

I would store the last processed watermark in a metadata table. ADF would extract records greater than this value from the source. Data would land in Bronze, be processed in Silver, and upon successful load, the watermark would be updated. This ensures reliable incremental ingestion.

54. What is Change Data Capture (CDC)?

CDC captures changes (inserts, updates, deletes) from a source system so downstream systems can replicate only modified records rather than full reloads.

55. How does CDC work conceptually?

CDC monitors transaction logs or change tables to identify data modifications. These changes are then streamed or extracted and applied to downstream systems incrementally.

56. If CDC is not enabled, how would you handle change tracking in Azure Data Factory?

If CDC isn't available, I use watermark-based incremental loads, timestamp comparisons, or hash-based change detection. ADF supports incremental extraction using query filters based on modification timestamps.

57. What alternative mechanisms would you implement for detecting incremental changes?

Alternatives include comparing hash values of records, using audit tables, leveraging surrogate keys with incremental logic, or implementing periodic delta comparisons between source and target datasets.

58. If you are required to process and load one trillion records, what architecture and strategy would you use?

At that scale, I would design a distributed architecture using ADLS as storage and Databricks with optimized partitioning and autoscaling clusters. Data would be processed incrementally in partitions, avoiding monolithic operations. Proper file sizing and partition pruning become critical.

59. How would you ensure scalability and performance at that scale?

I would focus on partition alignment, minimizing shuffle operations, optimizing file sizes, enabling Adaptive Query Execution, and separating workloads. Infrastructure would be designed for horizontal scalability with cost-aware autoscaling.

60. Suppose your organization maintains a calendar table with working days and holidays. How would you design a pipeline or job that runs only on official working days?

I would integrate the calendar table into the orchestration layer. Before pipeline execution, a lookup would check if the current date is marked as a working day. The pipeline proceeds only if true. This ensures business-aligned scheduling.

61. A business user complains about missing or spaced data during a live customer call — what would be your immediate response?

First, I would remain calm and acknowledge the issue. I would quickly validate data freshness and check whether the pipeline completed successfully. If needed, I would compare row counts or metrics between Silver and Gold layers. Immediate focus is restoring confidence and identifying whether it's a data delay, transformation issue, or reporting filter mismatch. Communication and quick validation are critical in such high-pressure moments.

Below is the **consolidated notes in Question & Answer format**, without missing a single word, starting from your introduction up to:

🔪 **“Is there a separate support/operations team that handles production failures...”**

Introduction

Q: Can you introduce yourself?

I am Sudheer Kumar Musalimadugu, a Lead Data Engineer with over 11 years of IT experience, specializing in cloud-native data engineering, automation, and analytics solutions.

Hi, I'm Sudheer Kumar Musalimadugu. I have over 11 years of IT experience, with around 6+ years focused specifically on Data Engineering and 5+ years in DevOps and automation.

Currently, I'm working with Kahramaa – Qatar General Electricity and Water Corporation, where I'm part of a large-scale data transformation and self-service analytics initiative.

My role involves designing and building end-to-end Azure-based data platforms using Azure Data Factory, Azure Data Lake Gen2, Databricks, Delta Lake, and Azure Synapse.

We handle both batch and real-time streaming data, including IoT sensor data, Oracle ERP systems, and other enterprise sources. I've implemented a Lakehouse architecture with Bronze, Silver, and Gold layers, ensured governance using RBAC and Unity Catalog, and built scalable Power BI solutions for business users.

Before that, at Kyndryl, I worked as a Lead Data Engineer on a cloud automation project for National Australia Bank, where I designed a Lakehouse architecture and built optimization workflows using PySpark and OR-Tools to automate network change management.

Before Kyndryl, I was a Senior Data & Automation Engineer at Empower Retirement (Feb 2021 – Mar 2022), where I modernized legacy data workflows and built scalable ingestion pipelines using Databricks Auto Loader, Delta Lake, and Azure Data Lake Storage. I developed complex real-time transformation and analytics pipelines and introduced data quality checks and metadata governance with Unity Catalog.

Earlier in my career, I worked with IBM and SLK Software, where I built ETL pipelines, automation frameworks, and Oracle-based data solutions.

How does your usual day look like?

In my current role as a Lead Data Engineer, my day is a mix of operational monitoring, development, and team coordination.

I usually start my day by reviewing Azure Data Factory pipelines and Databricks job runs to ensure all batch and streaming processes completed successfully. Since we handle both ERP batch data and high-volume IoT streaming data, I monitor pipeline health, latency, and failures using Azure Monitor and Databricks logs. If something fails, I analyze the root cause and coordinate fixes quickly to avoid downstream impact.

From a development perspective, I spend time enhancing ETL pipelines in ADF, writing and optimizing PySpark transformations in Databricks, and implementing Delta Lake merge logic for incremental and idempotent loads. I also focus on performance tuning — using proper partitioning strategies, Z-ordering, and file size optimization to keep queries efficient in Synapse and Power BI.

On the collaboration side, I lead a team of 6 data engineers. We follow Agile methodology, so I actively participate in sprint planning, backlog grooming, and daily stand-ups. During sprint planning, I help break down high-level business requirements into technical tasks, estimate effort, and ensure proper task allocation based on team strengths.

I also review code changes, guide junior engineers on best practices, and ensure our solutions align with governance standards like RBAC, Unity Catalog, and CI/CD practices through Azure DevOps.

Additionally, I coordinate with business stakeholders and reporting teams to understand evolving requirements and ensure we deliver scalable, production-ready data solutions.

So overall, my day balances technical implementation, platform reliability, performance optimization, and team leadership to ensure smooth and efficient delivery.

Is monitoring manual every day?

Yes, but it's not completely manual checking every single day.

We have automated monitoring in place using Azure Monitor and Log Analytics. Alerts are configured for pipeline failures, SLA breaches, long-running jobs, and streaming delays. So if something critical happens, we receive notifications immediately through email or Teams.

That said, as part of operational discipline, I do a quick health review in the morning. I check:

Failed or partially succeeded ADF pipeline runs
Databricks job failures or high execution time
Streaming lag or checkpoint delays
Resource utilization like cluster CPU/memory

This usually takes around 10–15 minutes unless there's an issue.

If I notice something abnormal, I drill into:

ADF activity run logs
Databricks driver and executor logs
Spark UI for stage-level performance
Error stack traces

We follow an RCA approach — identify whether it's data issue, schema drift, resource bottleneck, or infrastructure issue.

So it's a mix of:

Automated alerting (proactive)
Short daily operational validation (preventive)
Deep troubleshooting only when required

That ensures platform reliability without wasting time manually checking everything every day.

 When a pipeline fails, what is your immediate next step?

When a pipeline fails, my immediate focus is to quickly assess impact and identify the root cause.

First, I check the failure notification from Azure Monitor or Databricks to understand which pipeline or job failed and whether it’s a critical SLA-bound process.

Next, I go into:

ADF Monitor tab → Check activity-level failure details

If it's Databricks → Review job run logs and Spark UI

Check error message and stack trace

Then I categorize the issue into one of these buckets:

- Data issue – schema drift, null constraint, unexpected format
- Resource issue – cluster memory/CPU exhaustion
- Connectivity issue – SHIR down, network timeout, API failure
- Logic issue – transformation bug or edge case

If it’s a transient issue (like temporary connectivity), I safely re-trigger the pipeline.

If it’s a data or logic issue, I:

- Fix the root cause
- Validate with test run
- Then reprocess the failed batch

For streaming pipelines, I verify checkpoint integrity before restart to avoid data duplication.

Finally, if the issue affects business users or SLA, I communicate proactively with stakeholders and log an RCA for recurring prevention.

So overall, my approach is:

Assess impact → Diagnose root cause → Fix safely → Reprocess → Prevent recurrence.

 Is there a separate support/operations team that handles production failures, or does the same development team manage both development and production support?

In our setup, we follow a DevOps model, so the same data engineering team handles both development and production support.

We don’t have a completely separate operations team. Since we build the pipelines, we also own their reliability in production. That gives us better accountability and faster resolution because the team already understands the architecture and business logic.

However, we do have structured processes in place:

Automated alerts via Azure Monitor

Defined SLA-based severity levels

Incident tracking in Azure DevOps / ServiceNow

RCA documentation for recurring issues

For critical production incidents, we follow an escalation matrix and communicate with stakeholders proactively.

So it’s not unstructured support — it’s DevOps ownership with proper monitoring, alerting, and incident management practices.

Below is the continuation of your **consolidated interview notes in Question & Answer format**, preserving your exact wording and structure.

(Reference Resume:
Reference Intro Document:)

 Are you following any structured development methodology like Agile or Scrum?

Yes, we follow Agile methodology using Scrum framework.

We work in 2-week sprints. At the beginning of each sprint, we conduct sprint planning, where we break down business requirements into technical tasks — such as new data ingestion pipelines, schema changes, performance optimizations, or reporting enhancements.

Since I lead a team of 6 data engineers, I help:

- Refine backlog items
- Estimate story points
- Define technical approach
- Allocate tasks based on team strengths

We also have:

- Daily stand-ups to track progress and resolve blockers
- Sprint reviews with stakeholders to demonstrate completed pipelines or dashboards
- Retrospectives to improve performance, reduce recurring failures, and optimize delivery

All work items are tracked in Azure DevOps, including user stories, bugs, and production incidents.

This Agile approach helps us deliver incremental improvements while maintaining production stability and adapting to changing business requirements.

 Before building a pipeline from an on-prem database (like Oracle/MySQL), what are the key questions you ask?

1 Business Requirement & SLA

- Purpose of data
- Refresh frequency
- Data availability deadline

2 Data Volume & Growth

- Table size
- Daily incremental size
- Historical load requirement

3 Load Strategy

- Full vs Incremental
- Watermark column availability
- CDC enabled or not
- Primary key availability

4 Source System Impact

- Extraction window
- Read-only access?
- Can source handle parallel extraction?

5 Schema & Data Stability

- Schema volatility
- Data type consistency
- Known data quality issues

6 Connectivity & Security

- SHIR / VPN setup
- Firewall rules
- Encryption in transit
- PII or sensitive data handling

7 Error Handling & Reprocessing

- Retry mechanism
- Idempotent design
- Backfill strategy

 How do you design the architecture before starting implementation?

Before implementation, I follow a structured design approach covering five key areas: requirements, data flow, scalability, security, and operations.

1 Understand Requirements First

I start with:

- Business objective and use case
- Data freshness requirement (batch vs streaming)
- SLA and performance expectations
- Data retention requirements

This defines whether the architecture needs near real-time, micro-batch, or daily batch processing.

2 Analyze Data Characteristics

Next, I evaluate:

- Source type (Oracle, MySQL, APIs, IoT)
- Data volume and growth rate

Full vs incremental load
Schema volatility
Primary keys and watermark availability

This determines ingestion strategy and partition design.

3 Design the Data Flow (High-Level Architecture)

I then design the logical layers:

- Ingestion Layer – ADF / Event Hubs
- Landing / Raw Layer – ADLS Gen2
- Processing Layer – Databricks (Bronze → Silver → Gold)
- Serving Layer – Synapse / Power BI

I clearly define responsibilities of each layer and avoid mixing concerns.

4 Plan for Scalability & Performance

Before coding, I decide:

- Partition strategy (based on query patterns)
- File size targets
- Incremental merge logic
- Streaming configuration (checkpointing, micro-batch size)
- Expected data growth over 1–3 years

This avoids redesign later.

5 Security & Governance Design

I define:

- RBAC model
- Sensitive data masking
- Encryption in transit and at rest
- Unity Catalog structure
- Data lineage tracking

6 Monitoring & Failure Strategy

I design for reliability:

- Automated alerting
- Retry logic
- Idempotent pipelines
- SLA tracking
- Backfill mechanism

🔧 What Azure Data Factory activities have you used in your pipelines?

In my projects, I’ve used a mix of data movement, control flow, and orchestration activities in Azure Data Factory.

◆ Data Movement Activities

Copy Activity – Most commonly used, especially for ingesting data from Oracle/MySQL into ADLS Gen2.

Azure Databricks Notebook Activity – To trigger PySpark transformations in Databricks.

Stored Procedure Activity – For executing SQL logic in Synapse or source systems.

Web Activity – For calling REST APIs when integrating external systems.

◆ Control Flow Activities

ForEach Activity – For dynamic processing of multiple tables or partitions.

If Condition Activity – For conditional execution logic.

Switch Activity – When handling multiple source types.

Until Activity – For retry logic or polling-based workflows.

Wait Activity – To introduce controlled delays.

Set Variable / Append Variable – For dynamic parameterization.

Execute Pipeline Activity – For modular pipeline design and reusability.

◆ **Validation & Monitoring**

Get Metadata Activity – To check file existence, last modified time, or schema.

Lookup Activity – To fetch control table configurations.

Validation Activity – To validate file availability before processing.

◆ **Integration Runtime Usage**

Self-Hosted IR – For on-prem Oracle/MySQL connectivity.

Azure IR – For cloud-based sources.

Continuing your **complete consolidated interview notebook (Q&A format)**
— preserving your exact structure and wording —

🔪 **Bronze, Silver, Gold Layers – In Context of My Project**

In my Kahramaa project, we implemented a Lakehouse architecture in ADLS Gen2 using Databricks and Delta Lake.

Each layer has a specific responsibility.

👉 **Bronze Layer – Raw Ingestion Layer**

◆ **What Happens Here?**

This is the raw data landing zone.

Data from:

- Oracle ERP
- MySQL
- IoT sensors (streaming)
- APIs

Is ingested via:

- Azure Data Factory (batch)
- Event Hubs + Structured Streaming (real-time)

We store data in Delta format but:

- No heavy transformation
- Minimal schema enforcement
- Preserve original structure

◆ **Purpose**

- Maintain raw historical copy
- Enable replay/reprocessing
- Support audit and lineage
- Handle schema drift via Autoloader

◆ **Example in Your Project**

IoT sensor JSON events land as-is into Bronze tables partitioned by ingestion date.

👉 **Silver Layer – Cleansed & Standardized Layer**

◆ **What Happens Here?**

This is where transformation begins.

Using PySpark in Databricks, we:

Remove duplicates (using MERGE)
Handle null values
Standardize data types
Apply schema enforcement
Implement business rules
Handle late-arriving data
Create clean incremental tables

We also implement:

Watermark logic
SCD Type 1/2 where required
Delta ACID transactions

◆ **Purpose**

Produce trusted, clean datasets
Apply consistent structure
Ensure data quality

◆ **Example in Your Project**

Sensor readings are cleaned, invalid meter values filtered, timestamps standardized, and merged idempotently.

📊 **Gold Layer – Business & Analytics Layer**

◆ **What Happens Here?**

This layer is optimized for reporting and analytics.

We:

Aggregate fact tables
Join dimensions
Create star schema tables
Apply business KPIs
Optimize using Z-order
Partition based on query pattern

This layer feeds:

Azure Synapse
Power BI dashboards

◆ **Purpose**

High-performance queries
Department-specific datasets
Self-service analytics

◆ **Example in Your Project**

Meter reading fact table (billions of rows) joined with customer, region, and tariff dimension tables for Power BI consumption.

🎯 **Clean Summary for Interview**

Bronze → Raw, append-only, audit-ready ingestion layer.
Silver → Cleansed, deduplicated, standardized, business-rule-applied layer.
Gold → Aggregated, optimized, analytics-ready star schema layer for reporting.

In my project, this layered approach helped us handle 30–50 GB daily ingestion and scale up to tens of billions of records reliably.

🔧 **What transformations happen from Silver to Gold?**

The transformation from Silver to Gold is primarily business-focused modeling and optimization, not raw data cleaning.

In Silver, data is already cleansed, deduplicated, standardized, and validated.

From Silver → Gold, we perform:

1 **Business-Level Aggregations**

We aggregate granular data into business KPIs.

For example:

Hourly meter readings → Daily consumption
Daily → Monthly billing summary
Total energy consumption per region/customer

This reduces data volume and improves reporting performance.

2 Dimensional Modeling

We structure data into a star schema:

Create Fact tables (e.g., meter_reading_fact)
Create Dimension tables (customer_dim, region_dim, tariff_dim)
Generate surrogate keys
Implement SCD Type 2 where required

This makes it analytics-ready for Synapse and Power BI.

3 Business Rule Enrichment

We apply:

Derived columns (billing cycle, consumption category)
KPI calculations
Threshold logic (high usage alerts)
Data masking if needed

This is where business semantics are applied.

4 Performance Optimization

Since Gold is query-heavy, we:

Partition based on query pattern (e.g., by billing_date)
Apply Z-ordering on high filter columns
Optimize file size
Precompute aggregates to reduce BI query load

5 Data Access Structuring

We create:

Department-specific views
Access-controlled tables
Curated datasets for Power BI

 Strong Closing Line

So in summary:

Silver ensures data correctness and consistency.
Gold ensures business usability, performance, and analytics optimization.

Silver is engineering-focused.
Gold is business-focused.

 What kind of basic data cleansing do you perform in Bronze?

In the Bronze layer, we keep transformations minimal because its primary purpose is to preserve raw data for traceability and reprocessing.

However, we do perform basic structural and technical validations, not business-level cleansing.

The typical cleansing in Bronze includes:

1 Schema Normalization

Ensure correct data types (e.g., cast string timestamps to proper timestamp format)
Flatten nested JSON if required for consistency
Standardize column naming conventions

2 Corrupt Record Handling

Capture malformed or corrupt records into a separate “rescue” column (especially with Autoloader)
Log ingestion failures without stopping the pipeline

3 Basic Null & Format Checks

Validate mandatory technical fields like primary key or ingestion timestamp
Remove completely empty records

4 Add Technical Metadata Columns

We enrich Bronze with:

ingestion_timestamp
source_system
file_name
load_date

This helps with auditing and reprocessing.

5 Deduplication (Light)

If duplicates are obvious at ingestion (like exact same file replay), we may apply minimal deduplication based on primary key + ingestion timestamp.
But heavy deduplication happens in Silver.

 How do you handle null values in your pipeline?

It depends on the layer and business impact.

 In Bronze

We don’t aggressively modify nulls.
We only:

Validate mandatory technical fields (like primary key or ingestion timestamp).
Capture bad records separately if critical fields are null.

Bronze preserves raw integrity.

 In Silver

This is where we properly handle nulls:

If a field is mandatory for business logic → filter or reject the record.

If nullable but required for analytics → apply default values (e.g., 0 for numeric, “Unknown” for dimensions).

Use coalesce() in PySpark where appropriate.

For joins, handle null keys carefully to avoid data loss.

 In Gold

We ensure:

No nulls in fact foreign keys.

Dimension tables use surrogate keys with “Unknown” records.

Aggregations handle null safely using proper functions.

 Do you drop rows with nulls?

Not blindly.

It depends on the column and business criticality.

If a mandatory key field (like primary key or event timestamp) is null → we reject or quarantine the record.

If it’s a non-critical attribute → we retain the record and handle nulls using default values or business rules.

For dimension tables, we usually map null foreign keys to an “Unknown” surrogate key instead of dropping rows.

Dropping rows is always a last resort and only done when the data cannot be trusted or would break downstream integrity.

So null handling is rule-based, not generic filtering.

Continuing your **complete consolidated interview notebook (Q&A format)**
— preserving your exact wording —

 If the same country appears as IN, IND, India — how do you standardize it before Silver layer?

If the same country appears as “IN”, “IND”, and “India”, I standardize it before or during the Silver transformation using a controlled mapping approach.

First, I avoid hardcoding replacements directly in logic.

Instead, I create a reference mapping table, something like:

```
source_value | standardized_value
IN | India
IND | India
India | India
```

Then in the Silver layer, I join the incoming dataset with this mapping table and replace the source values with the standardized value.

This approach gives three advantages:

- It keeps transformation logic clean
- It allows business to update mappings without code changes
- It ensures consistency across all pipelines

If a new variation appears that is not in the mapping table, I route it to a data quality check for review.

So I standardize using reference-driven transformation, not hardcoded string logic, ensuring scalability and governance.

 Where would you store your mapping table?

I store mapping tables in the Silver layer as a Delta reference table, managed through Unity Catalog.

◆ Why Silver?

Because:

- It is standardized and curated.
- It supports controlled updates.
- It aligns with business-rule transformations.
- It is accessible for downstream joins.

Bronze is raw — not suitable.
Gold is analytics-focused — mapping belongs before that.

 Give a real business use case where you combined multiple Silver tables to create a Gold layer.

Yes, one real use case in our Kahramaa project was building a Gold layer for energy consumption and billing analytics.

In Silver, we had multiple cleaned and validated tables:

```
meter_readings_silver → cleaned IoT consumption data
customer_silver → standardized customer master data
tariff_silver → tariff rate structure
region_silver → geographical hierarchy
billing_cycle_silver → billing period metadata
```

All of these were independently cleansed, deduplicated, and validated in Silver.

◆ **In Gold Layer**

We combined these Silver tables to create:

 energy_consumption_fact

This fact table included:

- Customer surrogate key
- Region surrogate key
- Tariff key

Billing period
Total consumption
Calculated billing amount

We aggregated raw meter readings from hourly to daily or monthly level, depending on reporting requirement.

2 Dimension Tables

We also created:

customer_dim (with SCD Type 2 handling)
region_dim
tariff_dim

These were optimized for star schema design.

Business Outcome

This Gold model enabled Power BI dashboards for:

Regional energy usage trends
Revenue forecasting
High-consumption customer alerts
Tariff-based profitability analysis

It significantly improved performance because we pre-aggregated data instead of querying billions of raw records.

How many tables were involved?

In that specific use case, we combined around 5 to 6 Silver tables to build the Gold layer.

From Silver, we used:

meter_readings_silver
customer_silver
tariff_silver
region_silver
billing_cycle_silver

So roughly five core Silver tables were joined and transformed to create:

One primary fact table (energy_consumption_fact)

Three to four dimension tables (customer_dim, region_dim, tariff_dim, billing_cycle_dim)

So in total, around 8–10 tables were involved across Silver and Gold for that business use case.

What joins did you use?

In the Silver to Gold transformation, we primarily used left joins and inner joins, depending on the business requirement.

1 Inner Join

We used inner joins when the relationship was mandatory.

For example:

Joining meter_readings_silver with customer_silver on customer_id

If a meter reading doesn’t have a valid customer, it’s considered invalid data and handled earlier in Silver. So inner join was safe there.

2 Left Join

We used left joins when enrichment was optional.

For example:

Joining tariff details or region hierarchy.

If some enrichment data was missing, we still wanted the fact record, and we handled missing dimension keys using an “Unknown” surrogate key.

3 Aggregation Before Join

For performance, we first aggregated large datasets (like hourly readings → monthly totals) and then joined with dimensions. This reduced shuffle and improved performance.

🔑 **What was the final KPI delivered to the business?**

In that use case, the primary KPI delivered was Total Energy Consumption and Revenue per Customer and Region per Billing Cycle.

From the Gold layer, we enabled business to track:

- Total monthly energy consumption
- Billed amount based on tariff slabs
- Revenue per region
- High-consumption customer identification
- Consumption trend over time

One important derived KPI was:

Revenue per Unit of Energy, which helped analyze tariff effectiveness and regional profitability.

We also delivered:

- Peak vs off-peak usage trends
- Top 10 high-usage customers
- Region-wise consumption growth percentage

These KPIs were consumed via Power BI dashboards and helped departments with forecasting, load planning, and billing optimization.

🎯 Strong Closing Line

So technically we built fact and dimension models, but from a business perspective, we delivered consumption visibility, revenue insights, and performance tracking at a regional and customer level.

Continuing your **complete consolidated interview notebook (Q&A format)**
— preserving your exact content —

🔑 **Python Interview Snippet – Count Occurrences in a List**

Given a list like:

```
['A', 'B', 'A', 'C', 'B', 'A']
```

Return:

```
{'A': 3, 'B': 2, 'C': 1}
```

✅ **Python Solution (Recommended in Interview)**

```
from collections import Counter
```

```
data = ['A', 'B', 'A', 'C', 'B', 'A']
```

```
result = dict(Counter(data))
```

```
print(result)
```

Output:

```
{'A': 3, 'B': 2, 'C': 1}
```

💡 **Alternative (Without Counter)**

```
data = ['A', 'B', 'A', 'C', 'B', 'A']
```

```
result = {}
```

for item in data:

if item in result:

result[item] += 1

else:

result[item] = 1

print(result)

Both approaches return:

{'A': 3, 'B': 2, 'C': 1}

🎯 For interviews, mention Counter first (cleaner), then show manual logic if asked.

🔪 Remove Duplicates from a List

Given:

data = ['A', 'B', 'A', 'C', 'B', 'A']

✅ Method 1: Using set() (Order Not Preserved)

unique_list = list(set(data))

print(unique_list)

⚠️ Note: This does not preserve order.

✅ Method 2: Preserve Order (Recommended in Interview)

unique_list = list(dict.fromkeys(data))

print(unique_list)

Output:

['A', 'B', 'C']

✓ Preserves original order

✓ Clean and efficient

✅ Method 3: Manual Loop (If Asked to Avoid Built-ins)

unique_list = []

for item in data:

if item not in unique_list:

unique_list.append(item)

print(unique_list)

🔪 RDD vs DataFrame vs Dataset

◆ RDD (Resilient Distributed Dataset)

Think of RDDs as Spark’s original and low-level way to represent data. It’s basically a distributed collection of data spread across your cluster, where you have full control over how to process and transform the data. It’s powerful but kind of “manual” — you tell Spark exactly how to do each step. RDDs are fault-tolerant, meaning if part of the data is lost, Spark can recover it.

Analogy: Think of RDD as a big suitcase where you just throw in lots of things (data) without much order.

◆ DataFrame

DataFrames organize data into named columns and rows, like a SQL table. They support high-level operations like filtering and aggregation. Spark optimizes these operations using Catalyst optimizer.

Analogy: Like a well-organized filing cabinet with labeled sections.

◆ **Dataset**

Datasets combine type safety of RDD with optimization of DataFrame. Mostly used in Scala/Java.

Analogy: Filing cabinet with safety locks — structured + error checking.

◆ **Main Differences (Interview Summary)**

- RDD = Low-level control
- DataFrame = High-level, optimized, SQL-like
- Dataset = DataFrame + compile-time type safety (Scala/Java)

🔧 **How Does Apache Spark Work? (Architecture Overview)**

Apache Spark operates on a master-worker architecture.

It relies on two abstractions:

- Resilient Distributed Datasets (RDDs)
- Directed Acyclic Graphs (DAGs)

◆ **Driver Program**

Central coordinator of Spark application.

Responsibilities:

- Translate logical plan → physical plan
- Divide into stages and tasks
- Coordinate with cluster manager
- Monitor job execution

◆ **Cluster Manager**

Allocates CPU and memory resources.

Examples:

- Standalone
- YARN
- Mesos
- Kubernetes

◆ **Executors**

- Run tasks on worker nodes.
- Store cached data.
- Handle shuffle operations.

◆ **Execution Flow**

- 1 App submission
- 2 DAG creation
- 3 Stage division
- 4 Task scheduling
- 5 Execution on worker nodes

◆ **Spark Optimizations**

- Lazy Evaluation
- Data Locality
- In-memory Computing
- Speculative Execution

 How do you handle duplicate records when overlapping files arrive?

When a new file arrives containing duplicate entries from a previously processed file:

◆ **Deduplication Using Unique Keys**

Use unique identifier (e.g., transaction_id).

Compare and eliminate duplicates during merge.

◆ **Delta Lake MERGE INTO (Upsert)**

SQL Example:

MERGE INTO sales_data AS target

USING new_sales_data AS source

ON target.transaction_id = source.transaction_id

WHEN MATCHED THEN

 UPDATE SET *

WHEN NOT MATCHED THEN

 INSERT *

If record exists → Update

If not → Insert

Ensures no duplicate rows.

◆ **Idempotent Design**

Pipeline designed so that writing same data multiple times produces same final state.

Delta Lake supports:

ACID transactions

MERGE operations

Checkpointing

Version management

◆ **PySpark Example**

from delta.tables import DeltaTable

new_data = spark.read.format("json").load("/mnt/new_data")

delta_table = DeltaTable.forPath(spark, "/mnt/delta_table")

delta_table.alias("target").merge(

 new_data.alias("source"),

 "target.record_id = source.record_id"

).whenMatchedUpdateAll() \

.whenNotMatchedInsertAll() \

.execute()

 Strong Closing Line:

Duplicates from overlapping files are handled by merging based on unique keys rather than blindly appending data. This ensures data integrity and exactly-once processing.

 Power BI Connection Modes

Connection Type Comparison

Connection Type	Data Storage	Refresh Method	Transformation Support	Use Case
Import	In-memory in Power BI	Scheduled or manual	Full Power Query and DAX	Small to medium dataset, fast report UI
DirectQuery	Source system	Real-time queries	Limited Power Query and DAX	Large datasets, near real-time data access
Live Connection	External semantic model	Real-time queries	Model transformations outside	Enterprise semantic models reuse

 Row-Level Security (RLS) Example

Steps:

- 1 Under the **Modeling tab** → **Manage Roles** → **Create Role**

Example condition:

country = "Germany"

- 2 Select the appropriate table column
- 3 Write DAX expression filtering Germany
- 4 Publish to Power BI Service
- 5 In Power BI Service → Dataset → Click “3 dots” → Security
- 6 Add users to the role

Now only those users will see Germany-related data and dashboards.

 Data Volume Breakdown – Kahramaa Project

◆ **Real-Time Streaming Data (IoT Sensors)**

IoT Sensor Events per Device: 96 readings per day (every 15 minutes)
Total IoT Sensors: 100,000 smart water and electricity meters
Daily IoT Events: 9.6 million sensor events
Daily IoT Data Volume: 20–30 GB
Monthly IoT Data: 600–900 GB
Annual IoT Data: 7–10 TB
Individual Event Size: 2–3 KB per JSON event

◆ **Batch Data Sources**

Oracle ERP Database: 500 GB – 2 TB total size
Oracle ERP Daily Incremental: 5–15 GB
MySQL Daily Incremental: 3–8 GB
Total Daily Batch Ingestion: 8–23 GB

◆ **Combined Data Ingestion**

Daily Total: 30–50 GB
Monthly Total: 1–1.5 TB
Annual Total: 12–18 TB

◆ **Azure Data Lake Storage Gen2 Containers**

Landing Container

Size: 500 GB – 1 TB
Retention: 7–14 days
Access Tier: Hot Tier

Purpose: Raw ingested data before validation

Raw Container

Size: 5–10 TB
Retention: 3–6 months
Access Tier: Cool Tier

Purpose: Validated data with basic transformations

Cleansed Container

Size: 40–120 TB
Retention: 3–5 years
Access Tier: Hot (recent 30–90 days), Cool (historical)

Purpose: Analysis-ready transformed data

Total Data Lake Capacity

50–150 TB cumulative across all containers

◆ Azure Synapse Analytics Data Warehouse

Total Warehouse Size: 15–40 TB
Primary Fact Table Rows: 30–80 billion rows (meter readings)
Dimension Tables: Several million rows

◆ File Size Optimization Settings

Delta Lake Streaming Writes: 128 MB target file size
Delta Lake Batch Processing: 1 GB target file size
Minimum Partition Size: 1 GB

◆ Streaming Configuration

maxBytesPerTrigger: 2 GB per micro-batch
maxFilesPerTrigger: 500 files per trigger
Standard Trigger Interval: 5 minutes
Real-time Critical Interval: 1 minute

◆ Compression & Optimization

Compression Ratio: 5–10x (JSON → Parquet with Snappy)
Storage Reduction: 80–85%
Example: 100 GB JSON → 10–15 GB Parquet

🎯 Scale Perspective:

KB → Individual sensor events
MB → Streaming file size target
GB → Daily ingestion
TB → Monthly ingestion & warehouse size
PB → Future 10+ year retention scale

🔑 Database Objects Breakdown (326 Objects)

◆ Total Objects: 326

Source	Tables	Views	Functions	Procedures	Total	Volume
IoT Sensors	15	8	12	5	40	30 GB
Oracle ERP	45	25	18	35	123	5 GB
Network Telemetry	22	12	10	8	52	8 GB
ServiceNow	18	14	8	12	52	2 GB
Customer Data	12	8	5	6	31	3 GB

Source	Tables	Views	Functions	Procedures	Total	Volume
Vendor Feeds	8	4	3	2	17	1.5 GB
Weather API	5	3	2	1	11	0.5 GB
TOTAL	125	74	58	69	326	75 GB

◆ Key Insights

Tables: 125 (38%)
Views: 74 (23%)
Functions: 58 (18%)
Procedures: 69 (21%)

◆ Complexity Paradox

IoT Sensors: 40 objects for 30 GB → Simple, high-volume
Oracle ERP: 123 objects for 5 GB → Complex, low-volume
ServiceNow: 52 objects for 2 GB → Complex, tiny volume

🎯 Interview Tip:

Big data ≠ Complex objects.
ERP systems are logic-heavy.
IoT systems are volume-heavy.

🔑 Governance Architecture – Kahramaa Project

◆ Identity & Access

Azure Active Directory (AAD)
Single source of authentication
RBAC enforced across Synapse, Databricks, Power BI

◆ Data Governance Hub

Azure Purview
Scans:
ADLS Gen2
Databricks Unity Catalog
Azure Synapse
Power BI
Provides:
Catalog
Business glossary
Lineage tracking
Impact analysis

◆ Delta Lake Governance

ACID transactions
Schema enforcement
Time travel
OPTIMIZE / Z-ORDER
VACUUM

◆ Unity Catalog

Granular table-level permissions
Column-level security
Integrated with AAD

◆ **Operational Governance**

Azure Monitor → Alerts
Cluster Policies → Cost control
Backup & DR strategy

🎯 Final Strong Interview Closing Line:

We built not just a data pipeline, but a scalable, governed, secure, and performance-optimized enterprise data platform capable of handling 30–50 GB daily ingestion and tens of billions of records with reliability and auditability.
